

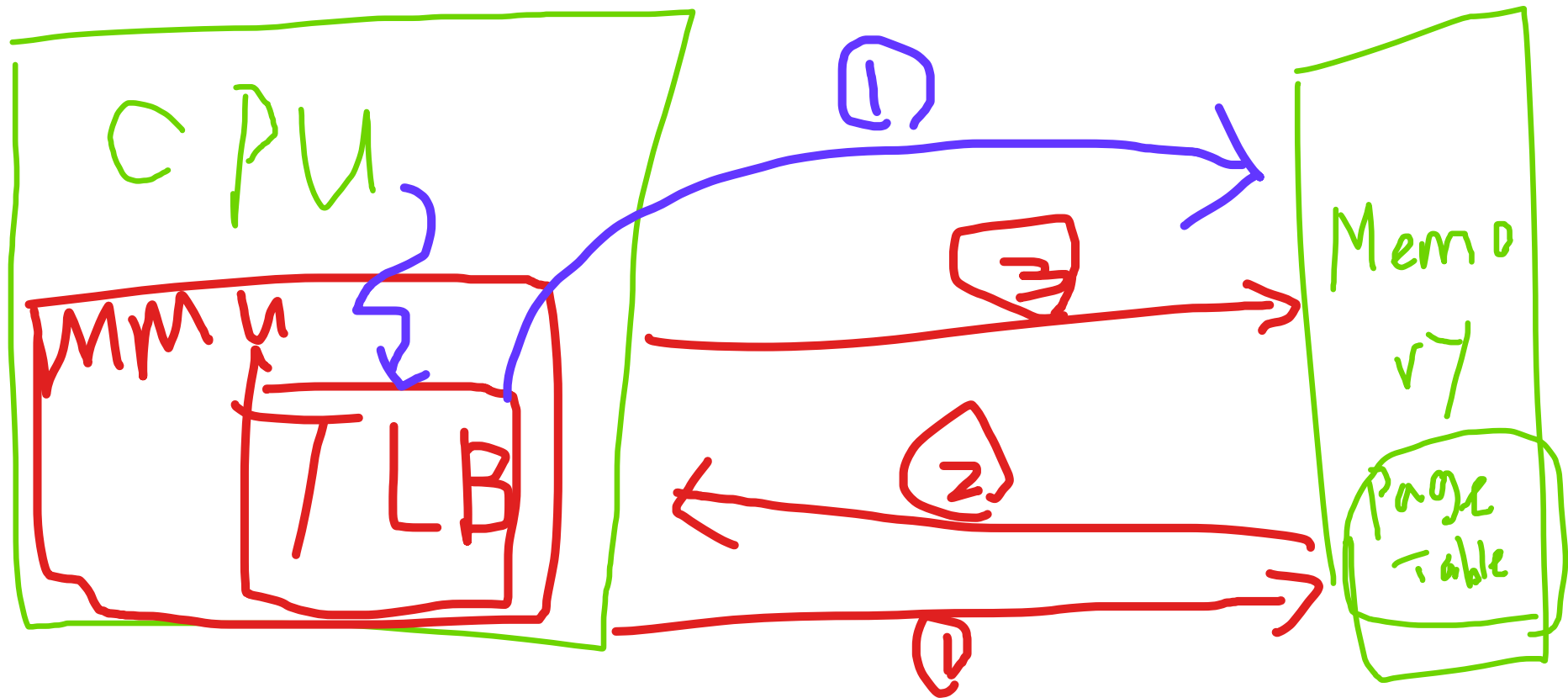
Memory Virtualization

- TLB and Advanced Page Tables -

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)



Translation Lookaside Buffer

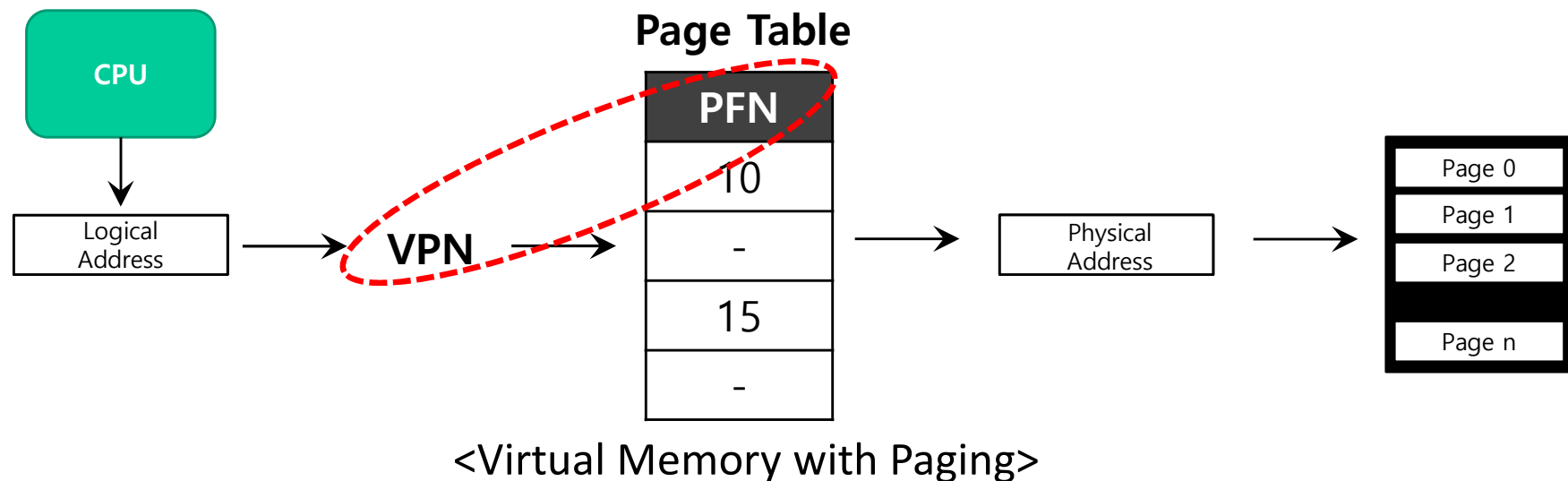
Translation Lookaside Buffer (TLB)

■ TLB

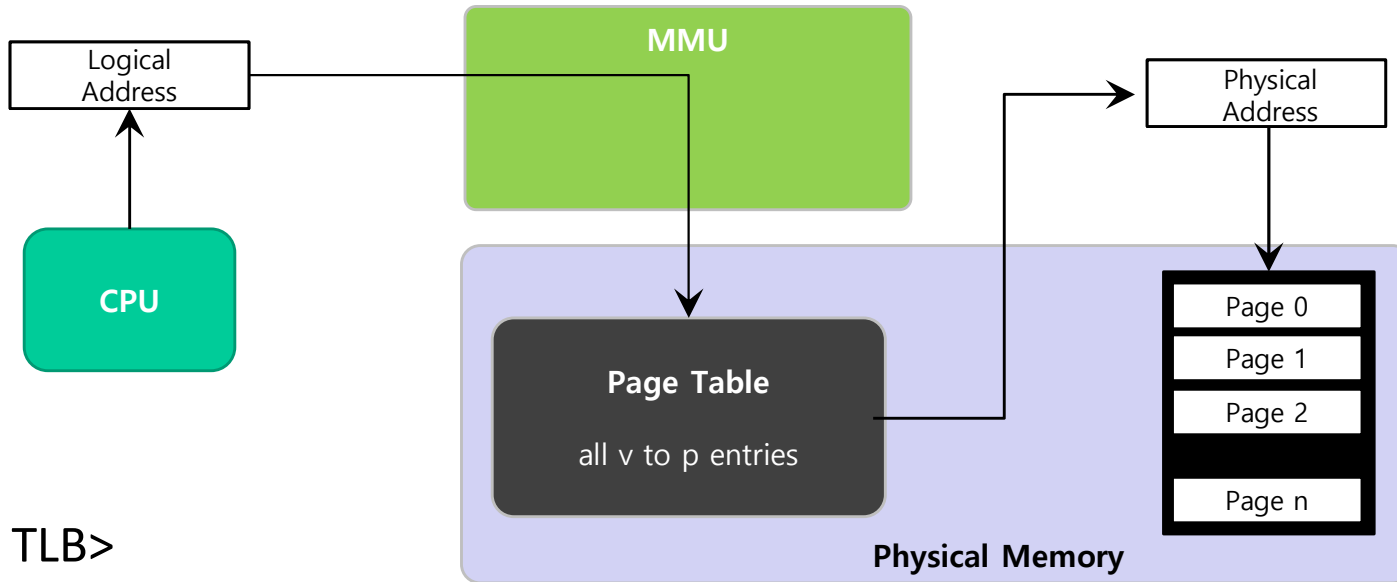
- A small, fast cache inside the Memory Management Unit (MMU).
- Stores recently used **virtual-to-physical address translations**.

■ Why Use a TLB?

- Without a TLB, each memory access would require a page-table lookup in memory.
- TLB hits allow the CPU to translate addresses in a single cycle, greatly improving performance.

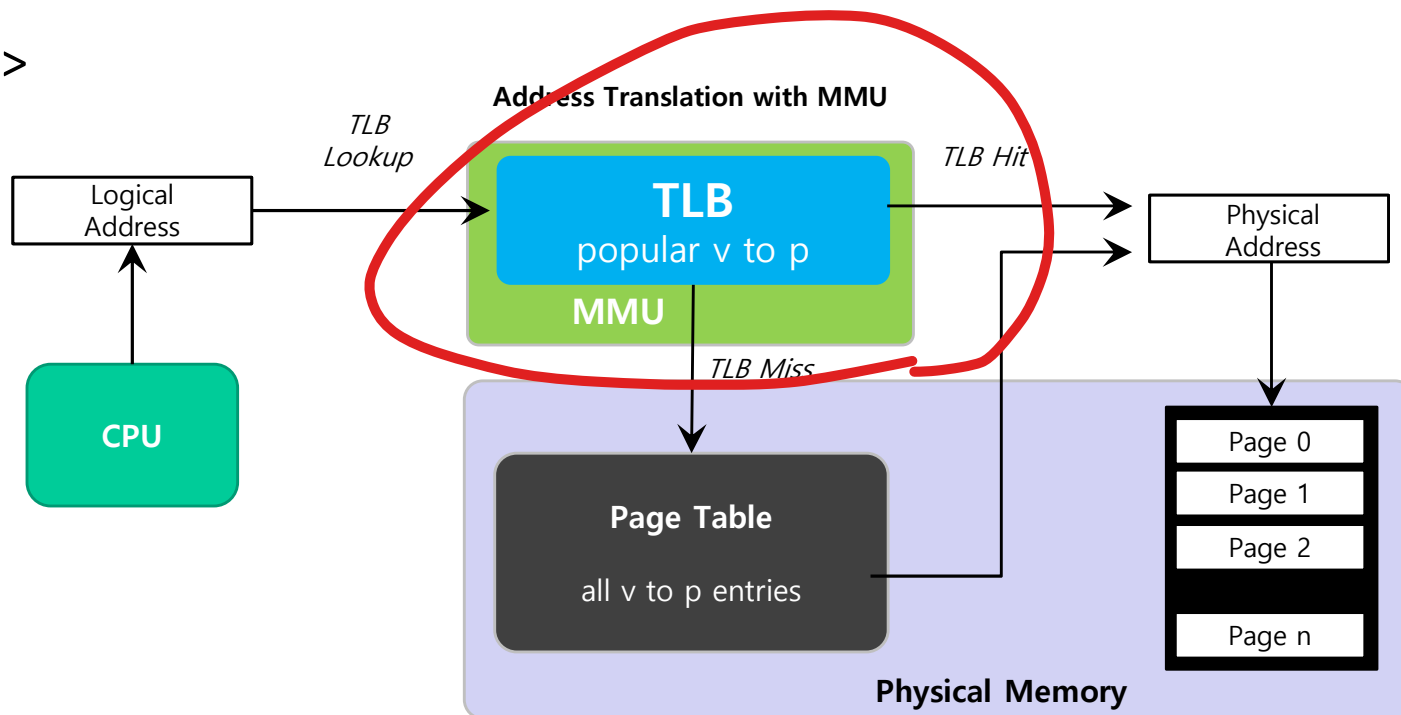


Address Translation with MMU



<Without TLB>

<With TLB>



*일부 페이지 테이블을 MMU에 들고있다.

TLB Basic Algorithms

```
1: VPN = (VirtualAddress & VPN_MASK ) >> SHIFT
2: (Success , TlbEntry) = TLB_Lookup(VPN)
3: if(Success == TRUE){ // TLB Hit
4:     if(CanAccess(TlbEntry.ProtectBit) == True ){
5:         offset = VirtualAddress & OFFSET_MASK
6:         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:         AccessMemory( PhysAddr )
8:     }else RaiseException(PROTECTION ERROR)
11: }else{ //TLB Miss
12:     PTEAddr = PTBR + (VPN * sizeof(PTE))
13:     PTE = AccessMemory( PTEAddr )
14:     if(PTE.Valid == False)
15:         RaiseException SEGFault) ;
16:     else{
17:         TLB_Insert( VPN , PTE.PFN , PTE.ProtectBits)
18:         RetryInstruction()
19:     }
```

- (11-12 lines) The hardware accesses the page table to find the translation.
- (16 lines) updates the TLB with the translation.

*TLB에 참조할 메모리 주소가 없다면 TLB를 우선 교체(insert) 후 retry 함.

Example: Accessing An Array

■ How a TLB can improve its performance.

	OFFSET				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

```
0:      int sum = 0 ;
1:      for( i=0; i<10; i++){
2:                  sum+=a[i] ;
3:      }
```

The TLB improves performance
due to spatial locality

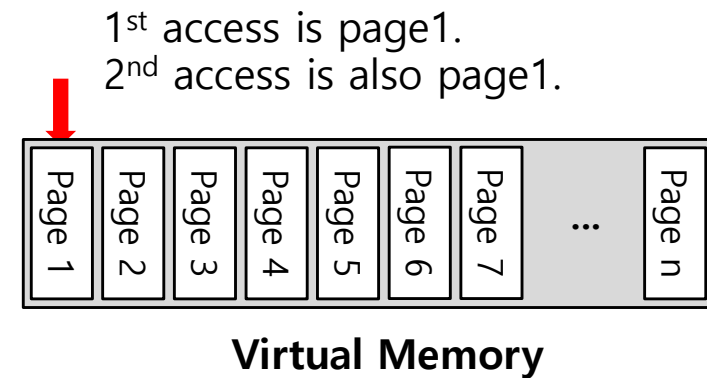
3 misses and 7 hits.
Thus **TLB hit rate** is 70%.

Locality

시간

■ Temporal Locality

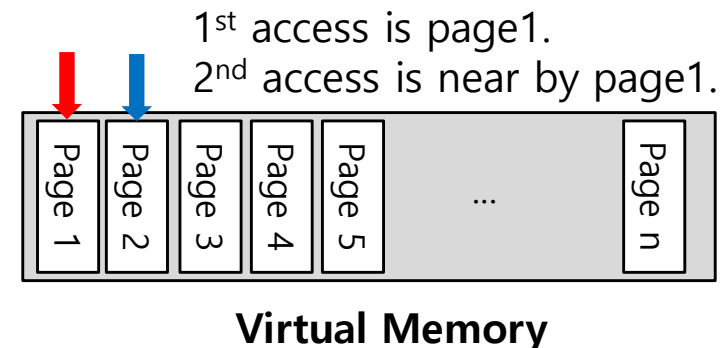
- An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



공간

■ Spatial Locality

- If a program accesses memory at address x , it will likely soon access memory near x .



Who Handles The TLB Miss?

■ **CISC** architectures: Hardware handles the TLB miss.

intel

- The hardware must know the exact location of the page tables in memory.
- On a TLB miss, the MMU **walks the page table**, finds the correct PTE, extracts the translation, **updates the TLB**, and **retries** the instruction automatically.
- This is known as a hardware-managed TLB.
- Example: **Intel x86**

■ **RISC** architectures: Software handles the TLB miss.

Arm

- On a TLB miss, the hardware raises a **trap** (exception) instead of walking the page table.
- The OS handles this trap in a **TLB miss handler**.
- The trap handler is kernel code written explicitly to:
 - locate the required page-table entry,
 - insert the translation into the TLB,
 - resume execution.
- This is known as a software-managed TLB.

CISC : 하드웨어 제어는 빠르지만 무거운 **Instruction**을 가짐.

RISC : 소프트웨어 제어는 조금 느릴 수 있지만 가벼운 **Instruction**을 가짐.

TLB Control Flow algorithm (OS Handled)

```
1: VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2: (Success, TlbEntry) = TLB_Lookup(VPN)
3: if (Success == True) // TLB Hit
4: if (CanAccess(TlbEntry.ProtectBits) == True)
5:     Offset = VirtualAddress & OFFSET_MASK
6:     PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:     Register = AccessMemory( PhysAddr )
8: else
9:     RaiseException(PROTECTION_FAULT)
10: else // TLB Miss
11:     RaiseException(TLB_MISS)
```

<Software-managed TLB>

<Hardware-managed TLB>

하드웨어적으로 한번에
해당 소스를 실행할 수 있음.

```
11: }else{ //TLB Miss
12:     PTEAddr = PTBR + (VPN * sizeof(PTE))
13:     PTE = AccessMemory( PTEAddr )
14:     if(PTE.Valid == False)
15:         RaiseException(SEGFAULT) ;
16:     else{
17:         TLB_Insert( VPN , PTE.PFN , PTE.ProtectBits)
18:         RetryInstruction()
19:     }
```

TLB Entry

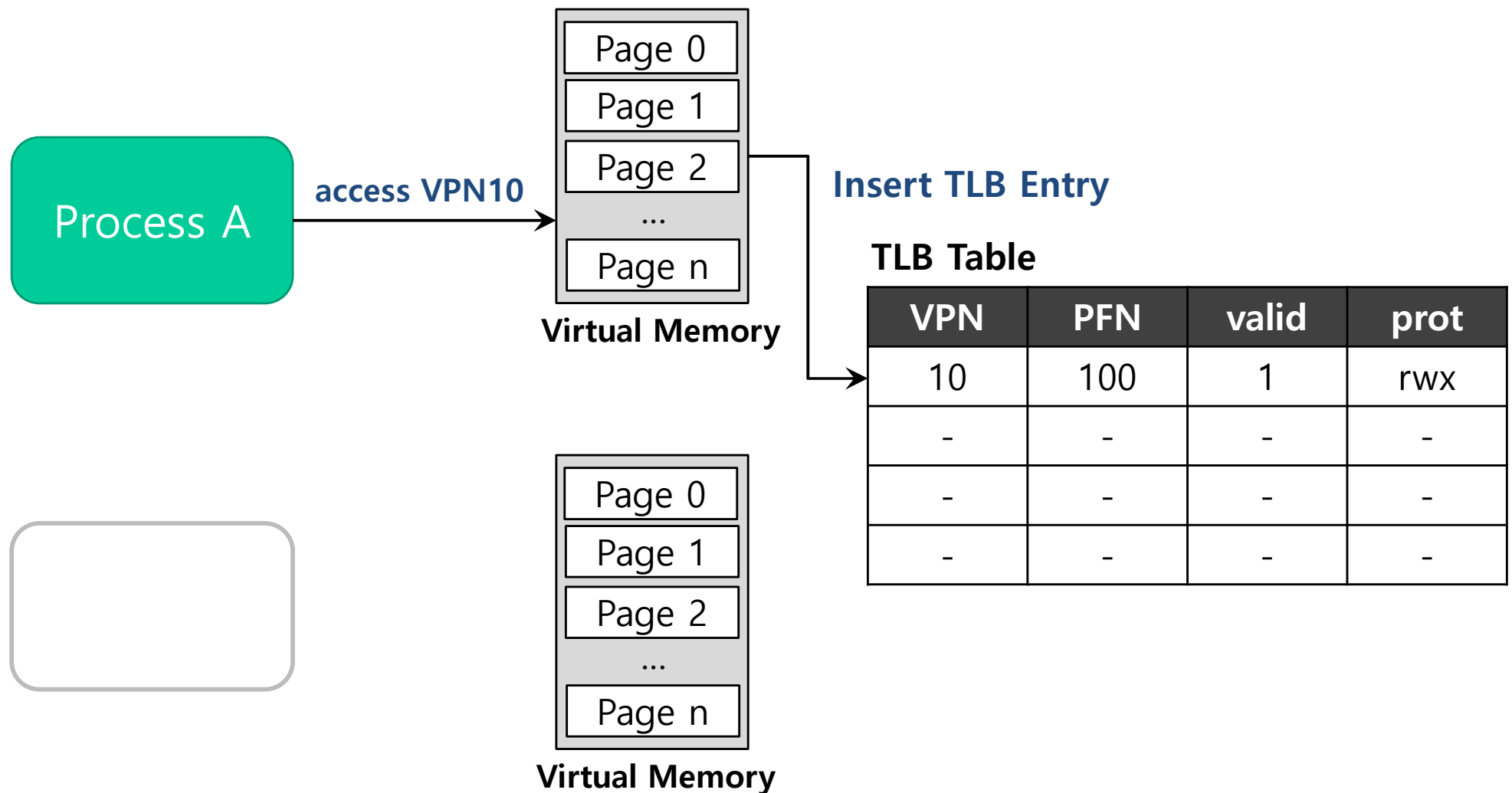
TLB entry에 접근하기 위한 캐싱 방법이 3가지가 있음.
1.fully - 아무곳에나 entry를 넣고 병렬 검색
2.set - 중간
3.direct - 순차적으로 entry를 넣고 mod계산

■ TLB is managed using a fully associative lookup.

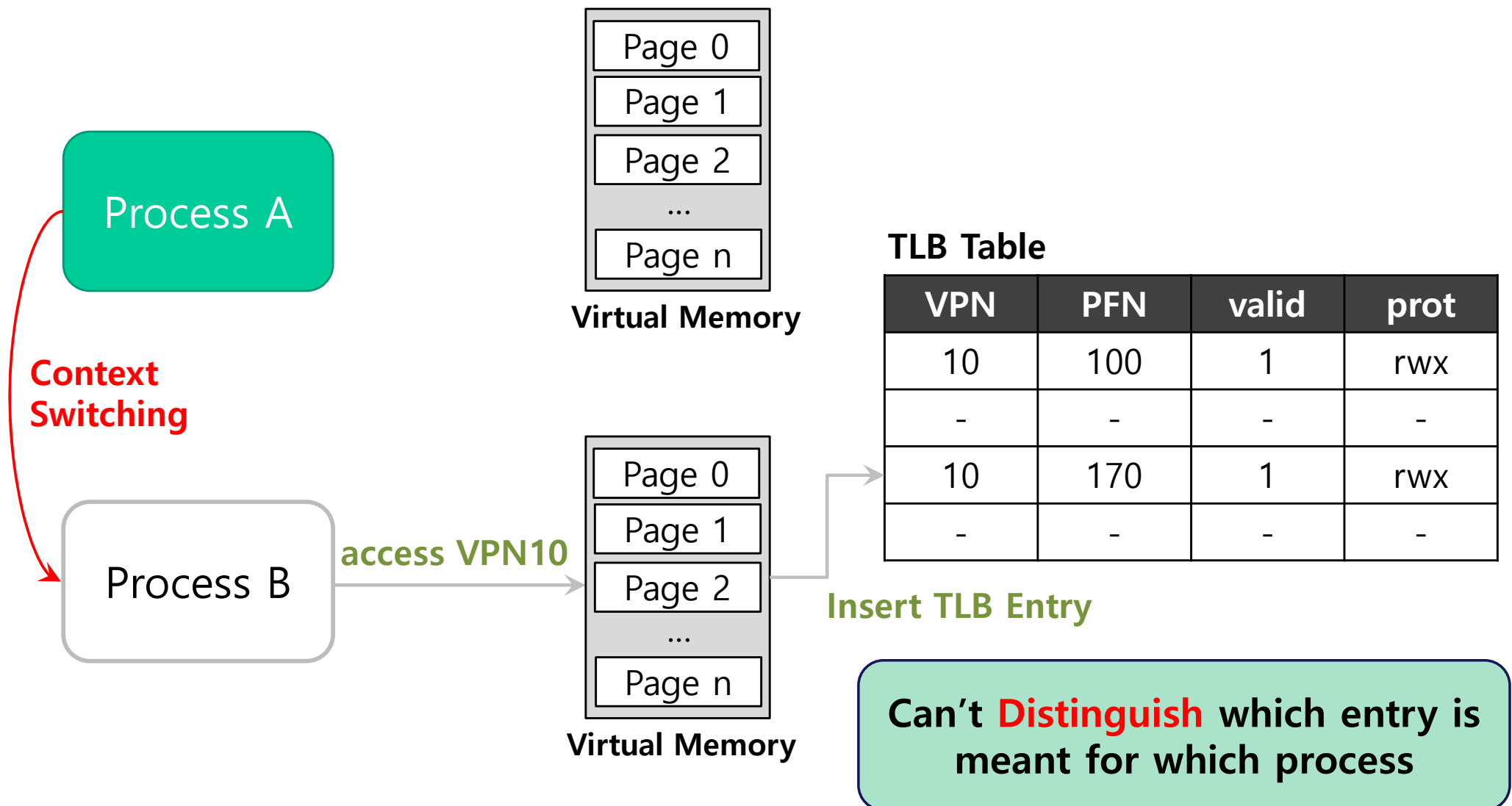
- A typical TLB contains **32, 64, or 128 entries** (sometimes more on modern CPUs).
- On every memory access, the **hardware searches** all TLB entries in parallel to find a matching VPN (virtual page number).
- Each TLB entry stores not only the translation but also several control bits:
 - valid bit,
 - protection bits (R/W/X),
 - address-space identifier (ASID),
 - dirty bit, etc.



TLB Issue: Context Switching

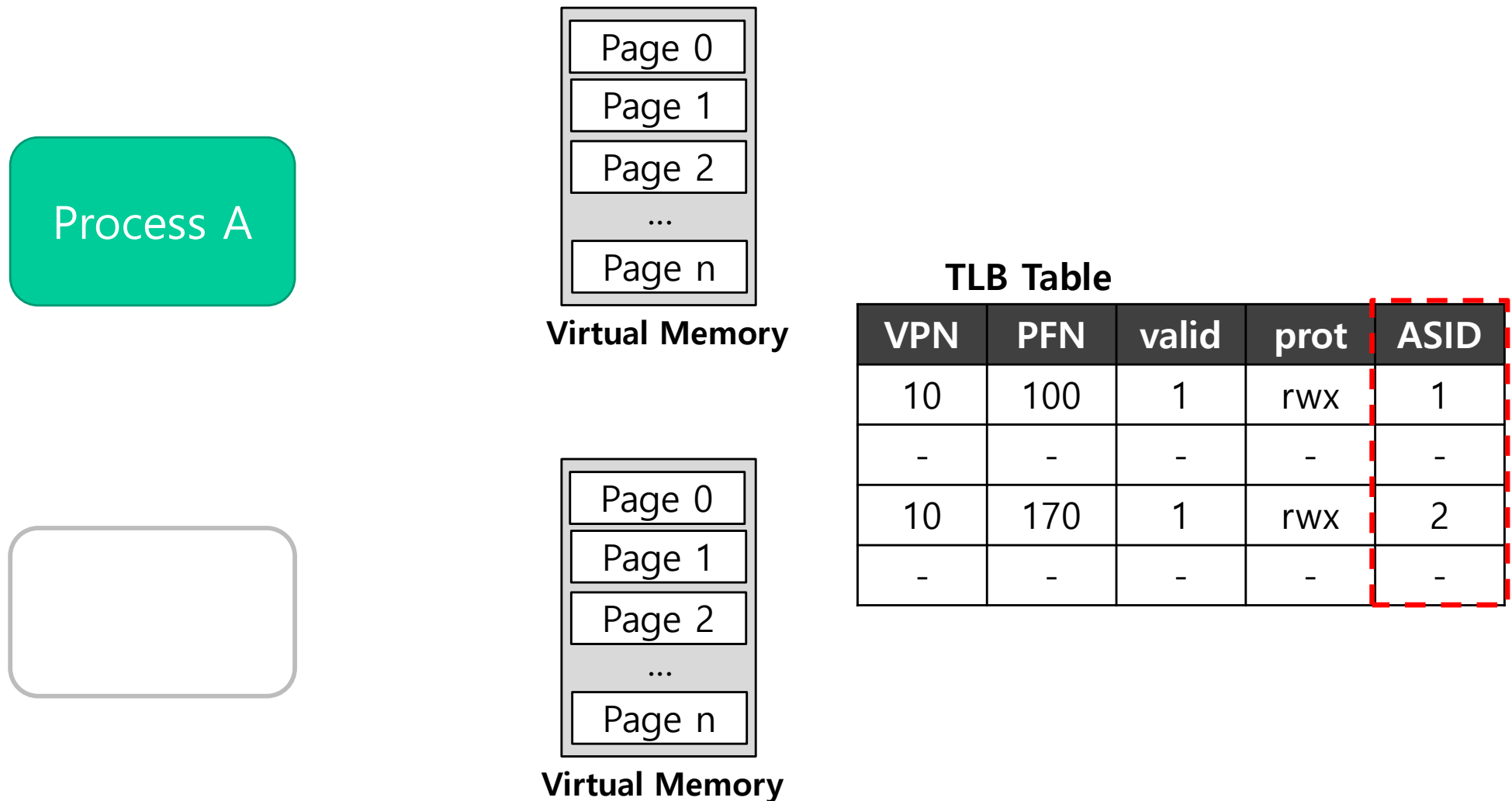


TLB Issue: Context Switching



To Solve Problem

- Provide an address space identifier(ASID) field in the TLB.



Another Case

■ Two processes **share a physical page**.

- Process 1 is sharing physical page **101** with Process2.
- P1 maps this page into the **10th** virtual page of its address space.
- P2 maps this page to the **50th** virtual page of its address space.

VPN	PFN	valid	prot	ASID
10	101	1	rwx	1
-	-	-	-	-
50	101	1	rwx	2
-	-	-	-	-

Sharing of pages is **useful** as it reduces the number of physical pages in use.

여러 프로세스가 동일한 라이브러리를 참조하는 등 중복되는 경우 동일한 물리 page를 참조할 수 있다.

■ Why is this useful?

- Sharing pages reduces the total number of physical frames required, thereby saving memory and enabling efficient inter-process sharing
 - (e.g., shared libraries, read-only text segments).

TLB Replacement Policy

■ LRU (Least Recently Used)

- Evicts the TLB entry that has **not been used for the longest time.**
- Exploits **temporal locality** in the memory-reference stream.

Reference Row

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1

TLB Entries:

7	7	7	2	2	4	4	4	0	1	1
	0	0	0	0	0	0	3	3	3	0
		1	1	3	3	2	2	2	2	2

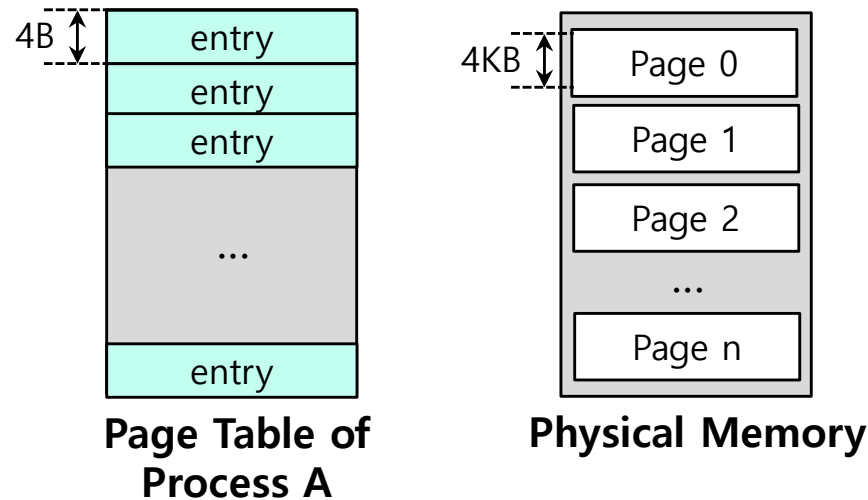
Total 11 TLB miss

Advanced Page Tables

Paging: Linear Tables

- **There is one page table per process.**

- Assume a 32-bit virtual address space, 4KB pages, and a 4-byte page-table entry.

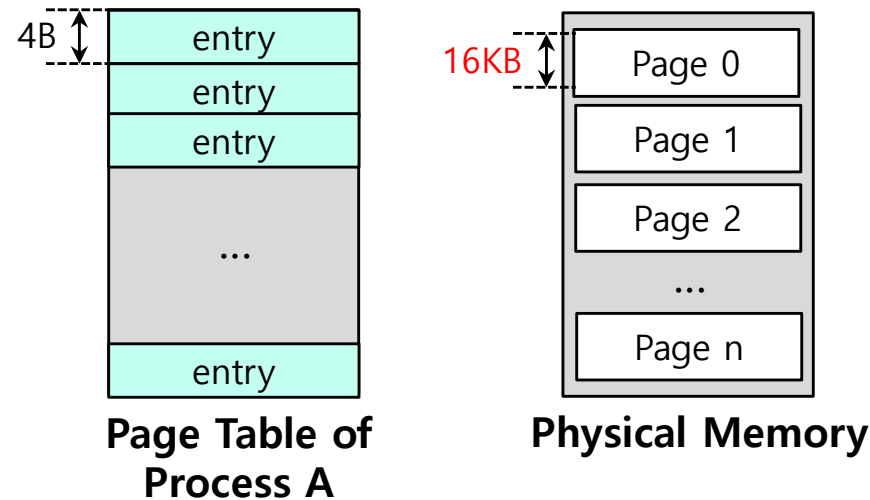


$$\text{Page table size} = \frac{2^{32}}{2^{12}} * 4\text{Byte} = 4\text{MByte}$$

Page tables are **too large**, leading to excessive memory consumption.

Paging: Smaller Tables

- Let's increase the page size from 4KB to 16KB!
 - Assume a 32-bit virtual address space with **16KB pages** and a 4-byte page-table entry.

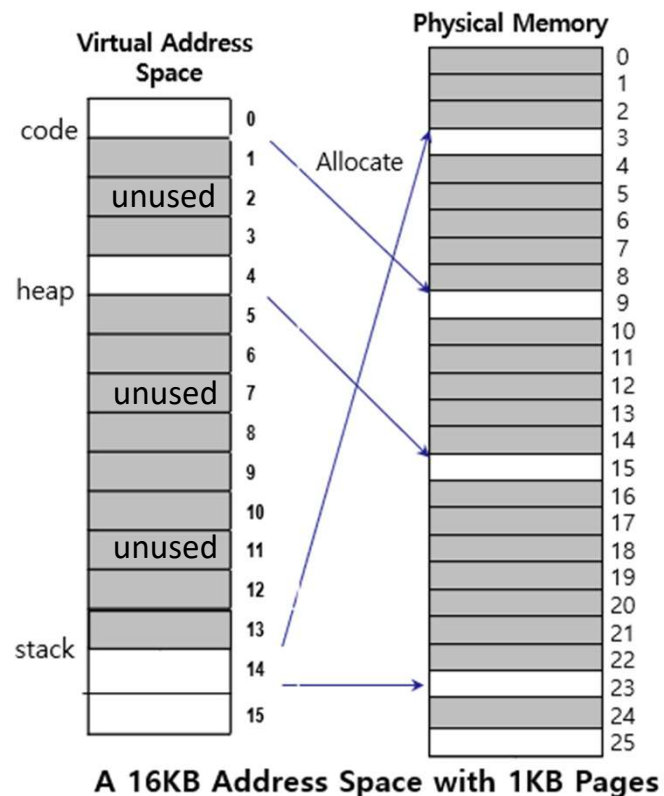


$$\frac{2^{32}}{2^{14}} * 4 = 1MB \text{ per page table}$$

Large pages lead to **internal fragmentation**.

Problem

- A single, linear page table represents the full virtual address space of a process.
- However, most entries in this page table are **unused** and remain invalid.



PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

unused

unused

A Page Table for 16KB Address Space

Multi-level Page Tables

■ Why multi-level paging?

- A single linear page table wastes memory because most entries are unused.
 - We need a structure that allocates page-table memory **only when needed**.

■ Key Idea: Turn the linear page table into a **tree-like structure**

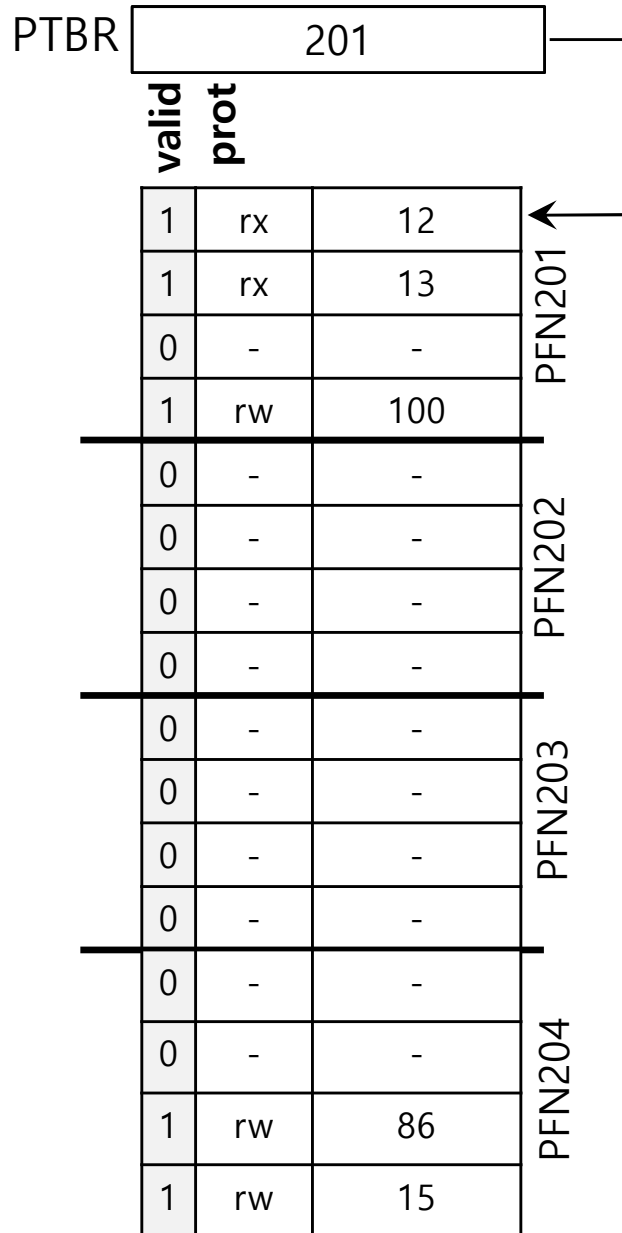
- **Divide the linear page table into page-sized chunks.**
- If an entire chunk (a page of page-table entries) is **invalid**, do **not allocate** it.
- Introduce a new structure—the **page directory**—to keep track of which page-table pages exist and which do not.

■ Page Directory

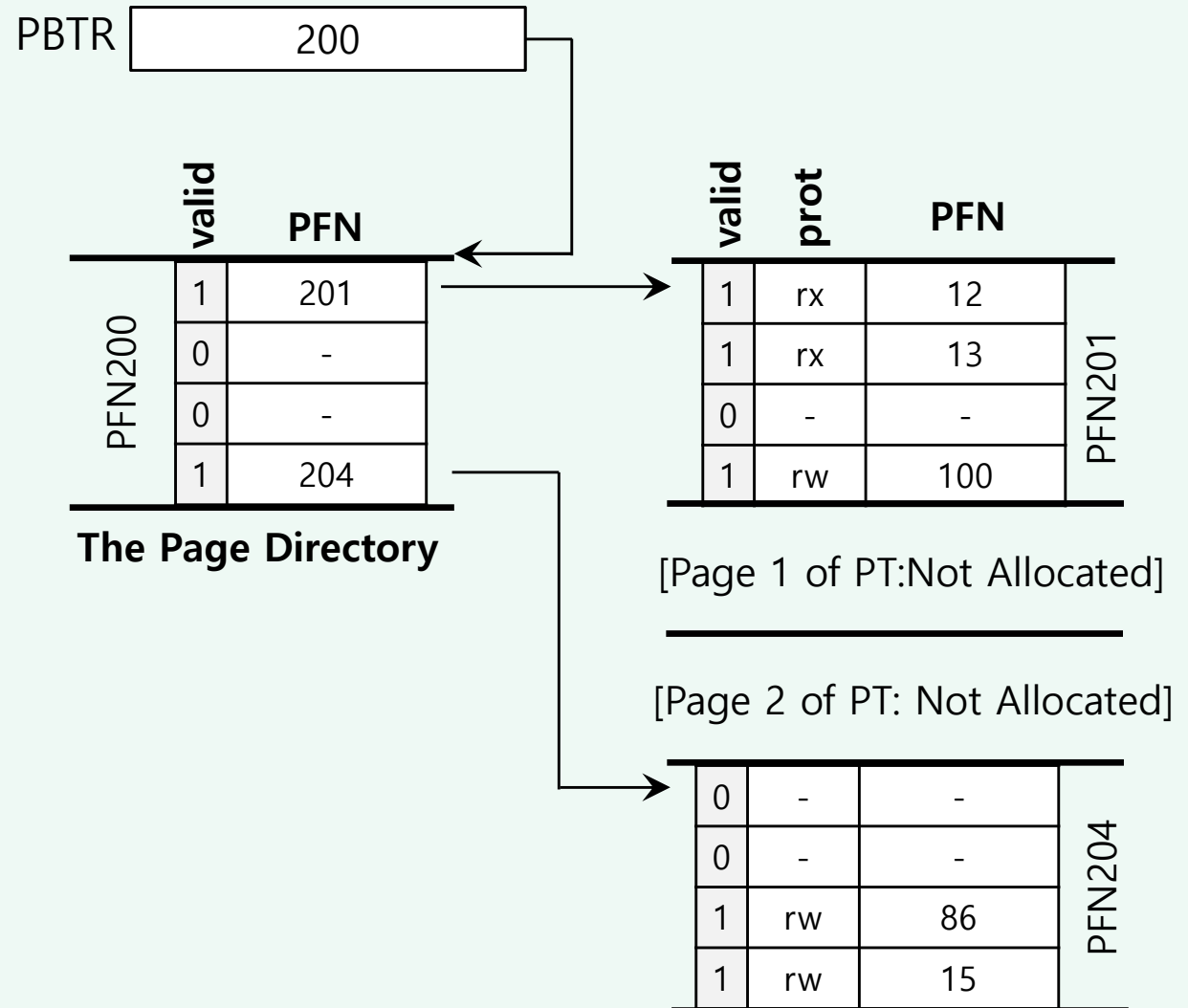
- Contains **one entry per page-table page**.
- Each **Page Directory Entry (PDE)** holds:
 - a **valid bit** (whether the page-table page exists),
 - the **PFN** of that page-table page.

Multi-level Page Tables: Page directory

Linear Page Table



Multi-level Page Table



Multi-level Page Tables

■ Advantages

- Page-table memory grows **only as needed**, proportional to the portion of virtual address space actually used.
- The OS can allocate a new page-table page **on demand** when memory use expands.

■ Disadvantages

- Increased **complexity** due to multi-level structure.
- Represents a classic time–space trade-off (**extra lookup steps** vs. reduced memory usage).

Example

0000 0000	code
0000 0001	code
...	(free)
	(free)
0000 0100	heap
0000 0101	heap
....	(free)
	(free)
1111 1110	stack
1111 1111	stack

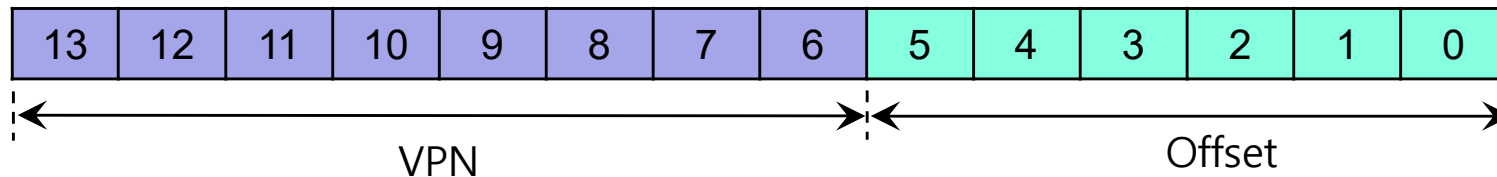
Process Virtual Memory

- **Page 0,1: code**
- **Page 4,5: heap**
- **Page 254, 255: stack**

Example

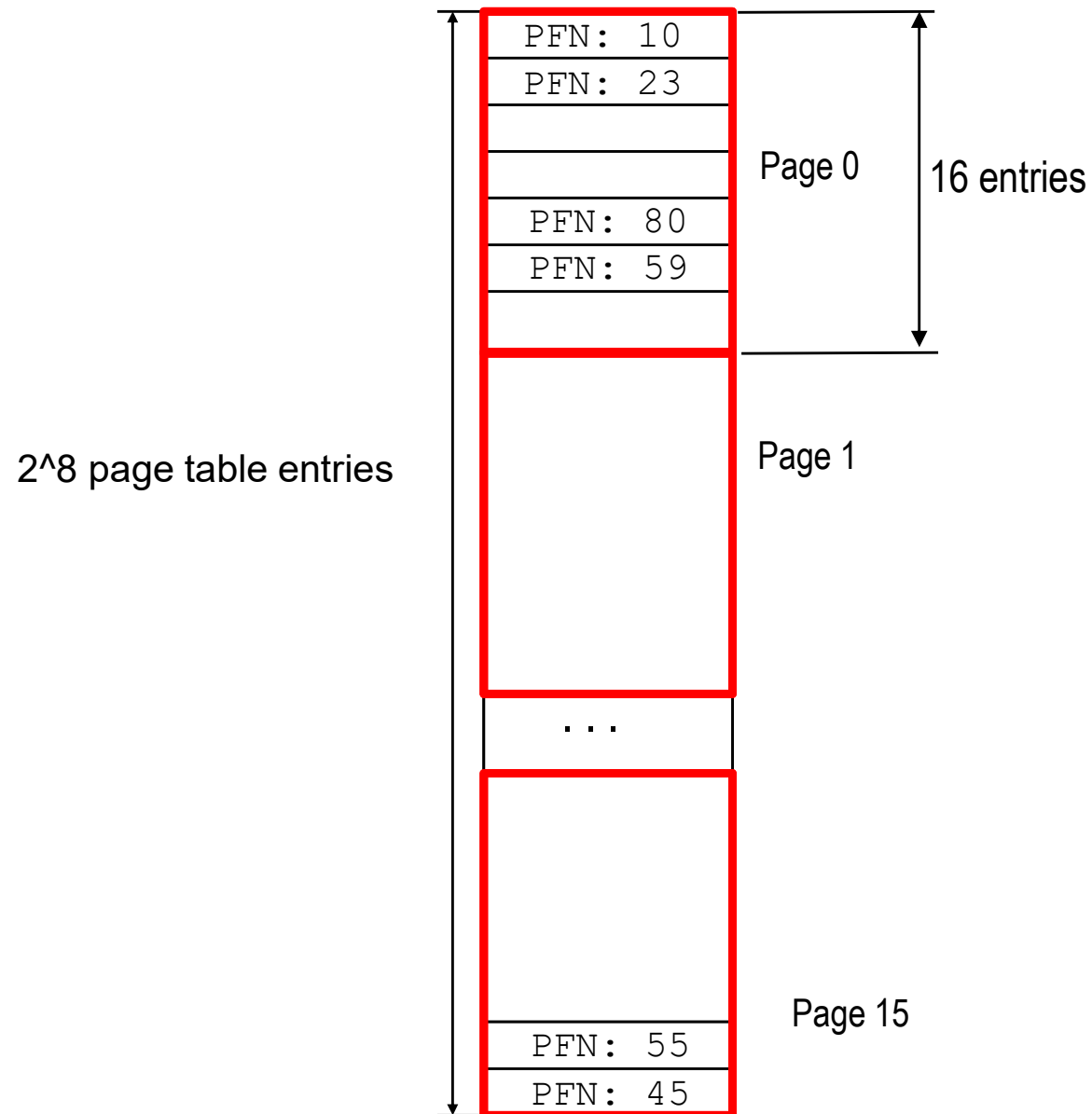
Flag	Detail
Address space	16 KB (2^{14} Byte)
Page size	64 byte
Virtual address	14 bit
VPN	8 bit
Offset	6 bit
Page table entry	4 Byte

A 16-KB Address Space With 64-byte Pages



- Single level paging
 - ◆ 256 page table entries: 2^8 entries
 - ◆ Page table size: $256 * 4 \text{ Byte} = 1 \text{ Kbyte}$
 - ◆ Sixteen pages (64 byte each): $1024/64 = 16$

Example: Single-level Page Table

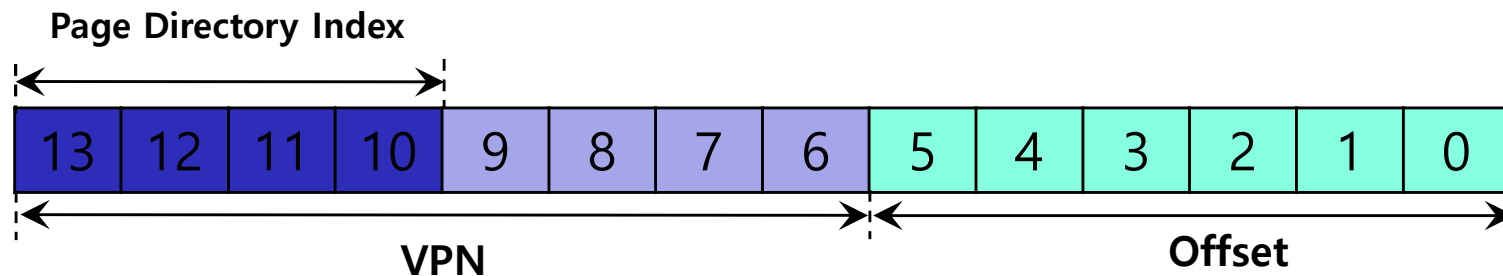


Example: Two-level Paging

■ Page Directory Index

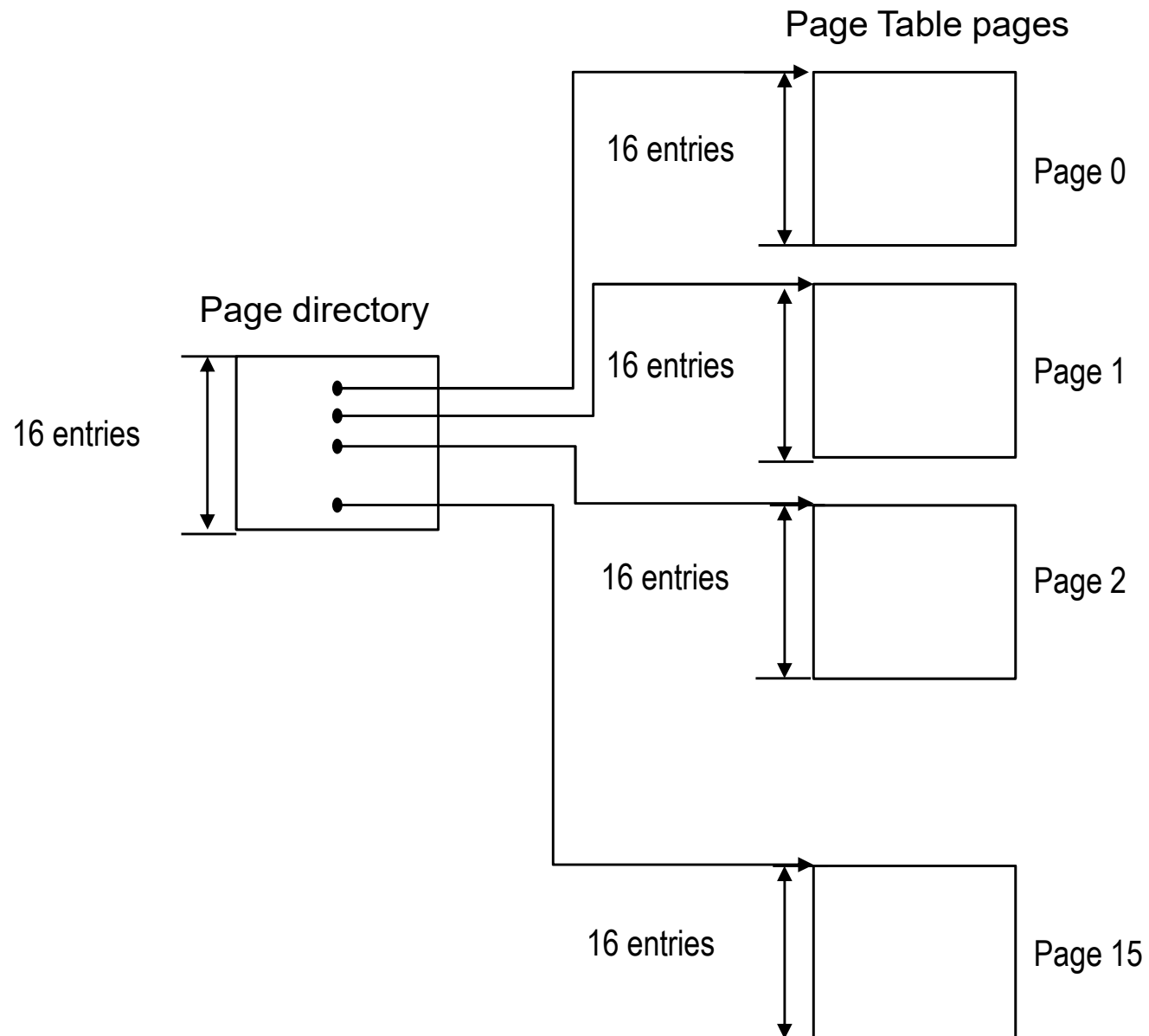
- A page table consists of 16 pages.
- 16 entries for page directory: one entry per page of the page table.
- $16 * 4 \text{ byte} = 64 \text{ byte}$ is required for page directory. → it can fit in a page.
- 4 bits for page directory index.

$$\text{PDEAddr} = \text{PageDirBase} + (\text{PDIndex} * \text{sizeof}(\text{PDE}))$$



- If the page-directory entry is **invalid**, raise an exception (The access is invalid).

Example: Two-level Paging

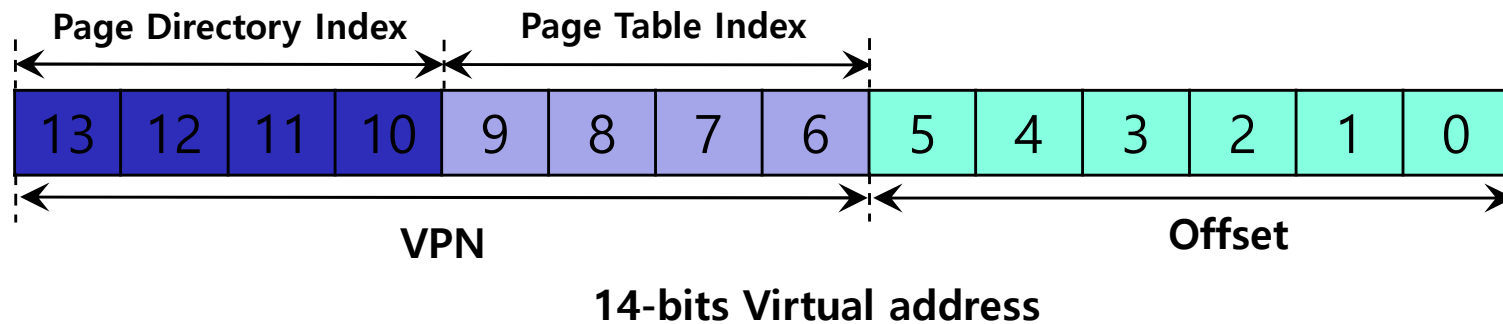


Example: Page Table Index

■ Page Table Index

- It is used to find the address of the page table entry.

$$\text{PTEAddr} = (\text{PDE.PFN} \ll \text{SHIFT}) + (\text{PTIndex} * \text{sizeof(PTE)})$$

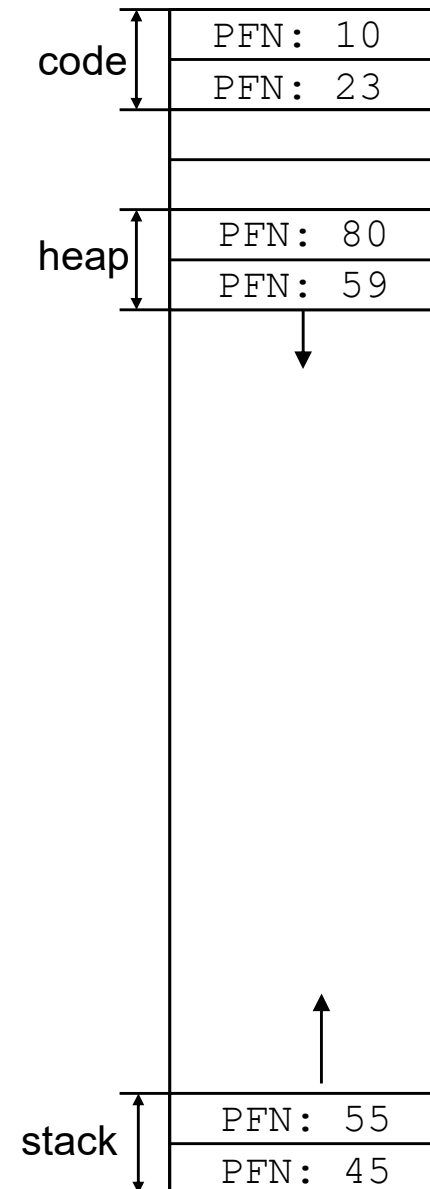


Examples

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

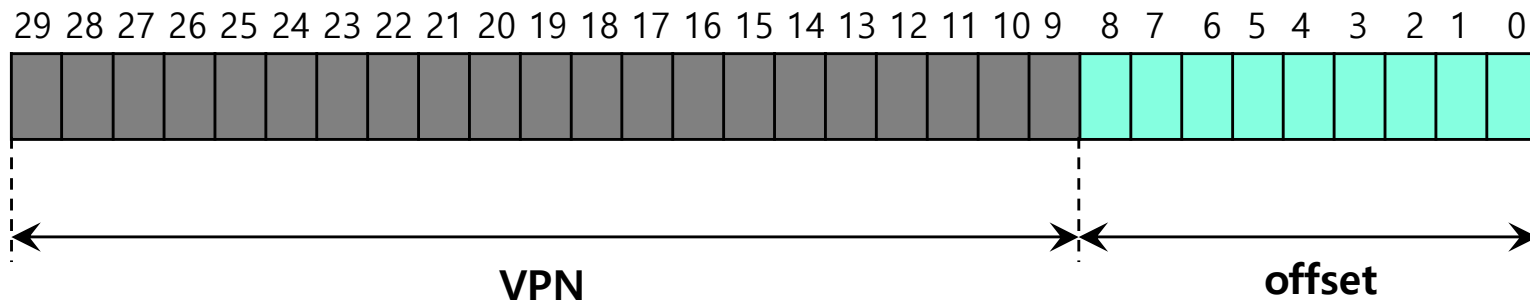
Single level paging: 16 pages

→ Two level paging: 3 pages



More than Two Levels

- In some cases, a deeper tree is required.
- Consider the following address space.



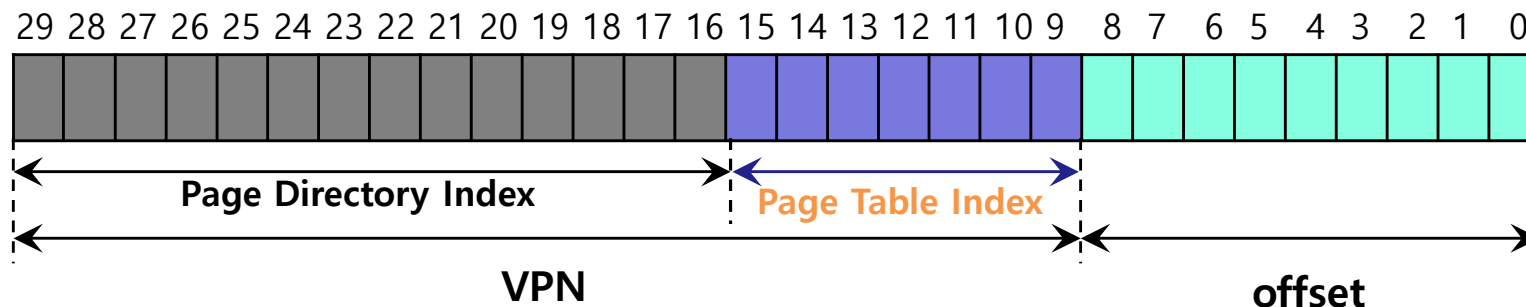
Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit

More than Two Level : Page Table Index

■ Number of pages in a page table

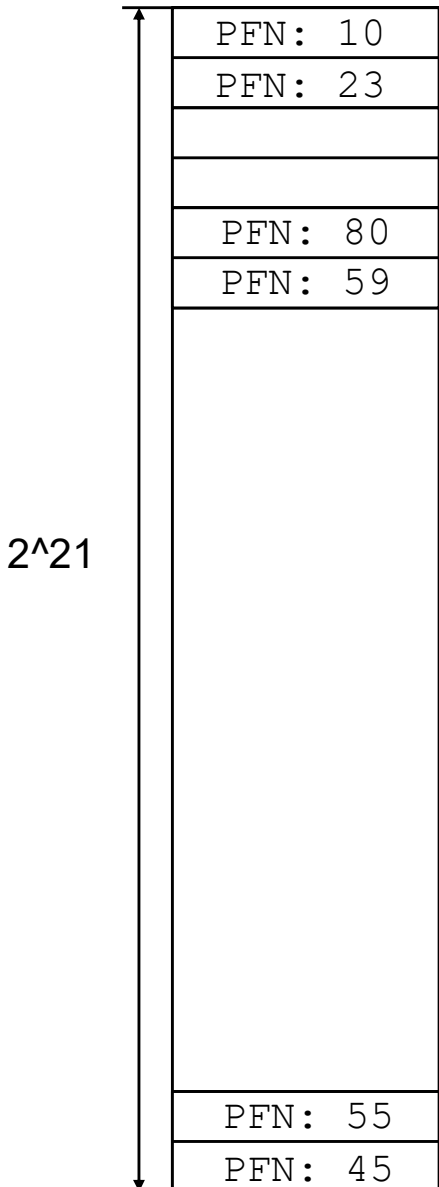
- 2^{21} page table entries
- 2^7 (128) PTE's in a single page (512/4, 512 page size, 4 byte pte size)
- 2^{14} pages in a page table

■ Number of page directory entries: 2^{14}

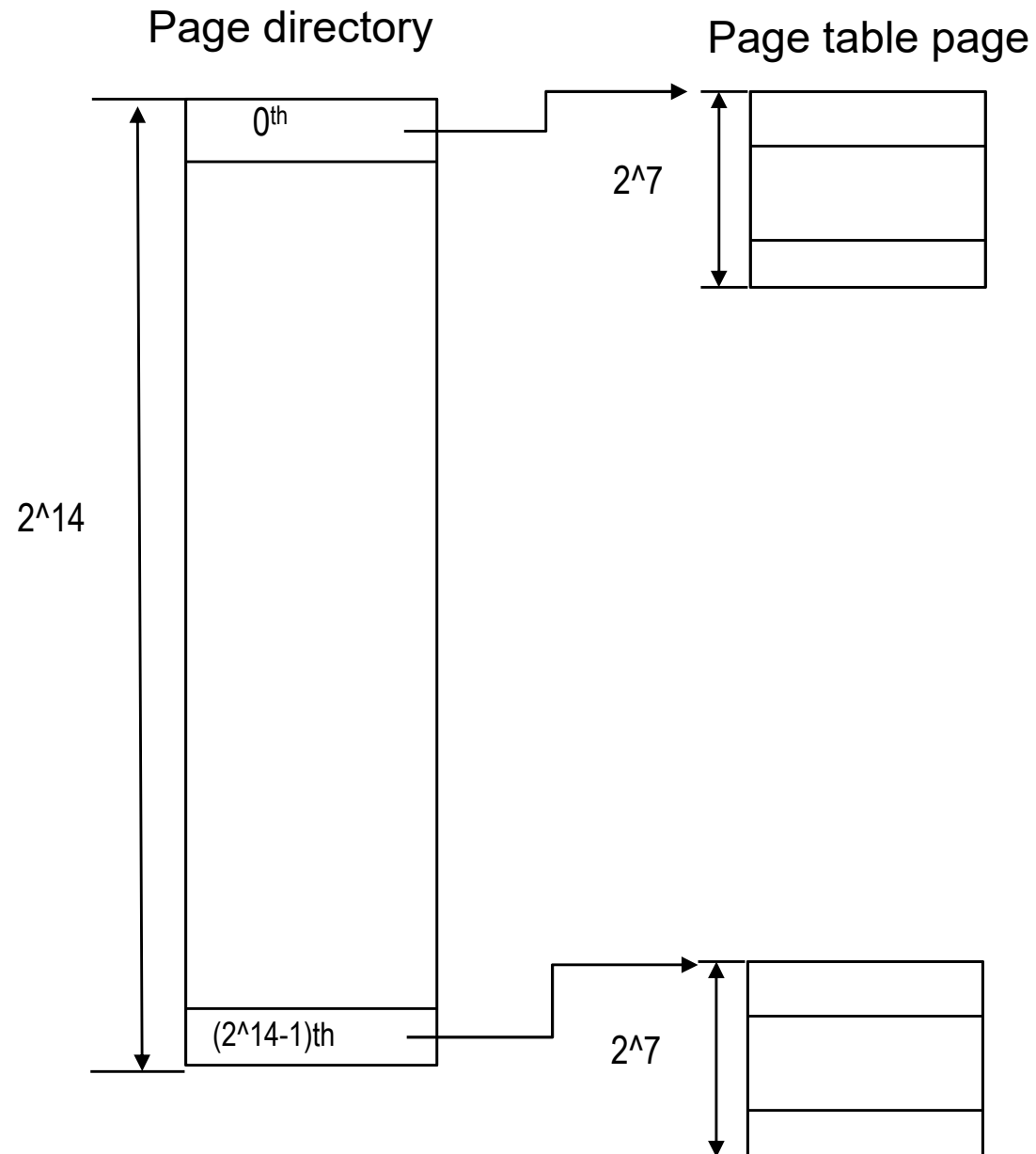


Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit
Page entry per page	128 PTEs (512/4)

Single-level Page Table

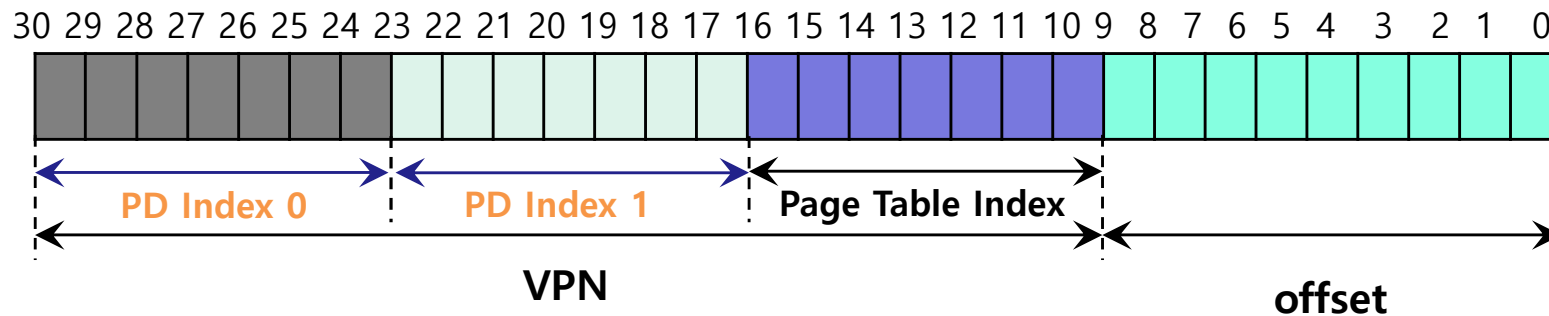


Two-level Page Table

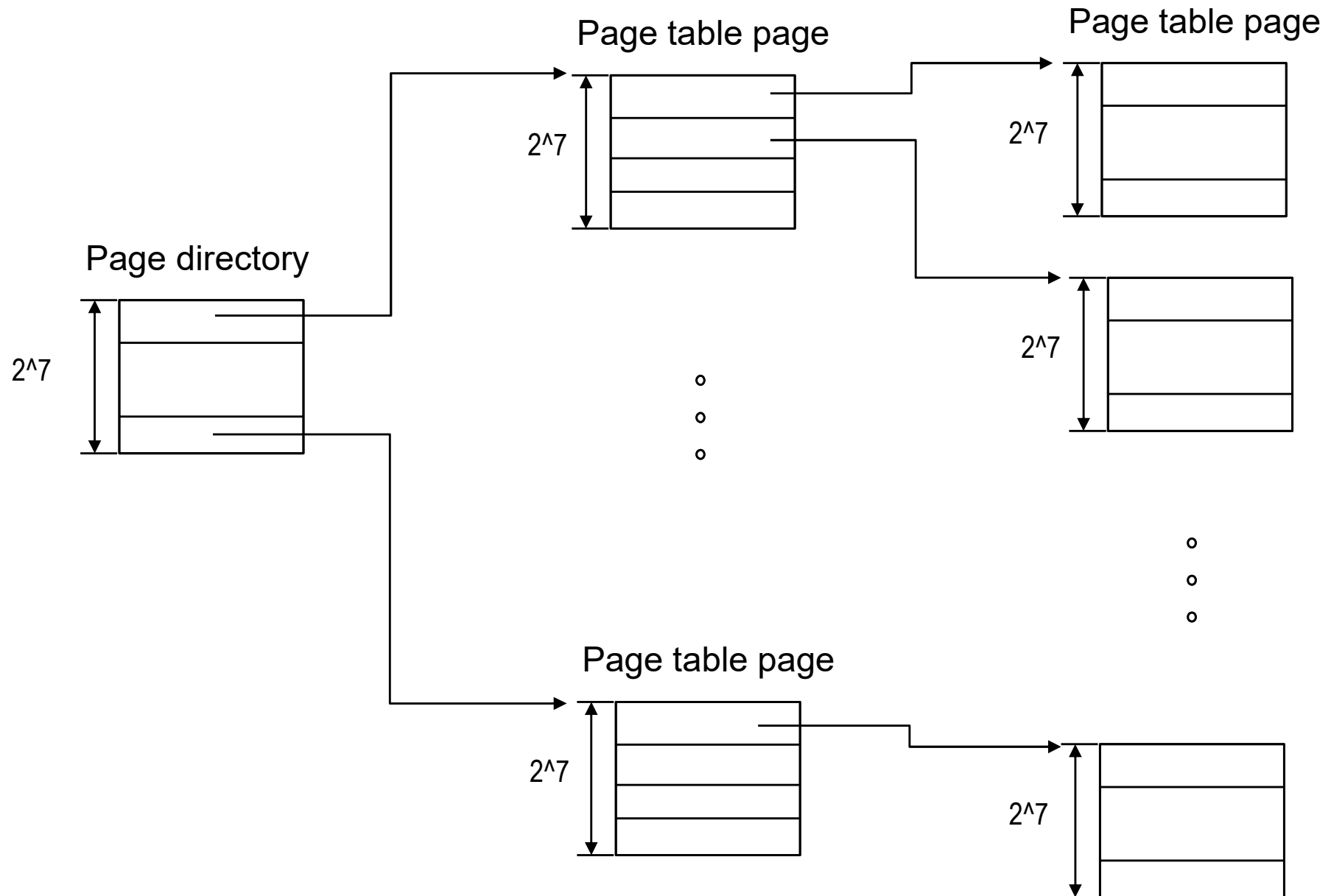


More than Two Level : Page Directory

- The number of pages in a page directory: $2^{14}/2^7 = 2^7$ (128)
- Page the page directory



Three-level Page Table



Multi-level Page Table Control Flow

```
01:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
02:      (Success, TlbEntry) = TLB_Lookup(VPN)
03:      if (Success == True)           //TLB Hit
04:          if (CanAccess(TlbEntry.ProtectBits) == True)
05:              Offset = VirtualAddress & OFFSET_MASK
06:              PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
07:              Register = AccessMemory(PhysAddr)
08:          else RaiseException(PROTECTION_FAULT);
09:      else // perform the full multi-level lookup
```

- ◆ (1 line) extract the virtual page number(VPN)
- ◆ (2 lines) check if the TLB holds the translation for this VPN
- ◆ (5-8 lines) extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory

Multi-level Page Table Control Flow

```
11:         else
12:             PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13:             PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14:             PDE = AccessMemory(PDEAddr)
15:             if(PDE.Valid == False)
16:                 RaiseException(SEGMENTATION_FAULT)
17:             else // PDE is Valid: now fetch PTE from PT
```

- ◆ (11 lines) extract the Page Directory Index(PDIndex)
- ◆ (13 lines) get Page Directory Entry(PDE)
- ◆ (15-17 lines) Check PDE valid flag. If valid flag is true, fetch Page Table entry from Page Table

The Translation Process: Remember the TLB

```
18:     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
19:     PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
20:     PTE = AccessMemory(PTEAddr)
21:     if(PTE.Valid == False)
22:         RaiseException(SEGMENTATION_FAULT)
23:     else if(CanAccess(PTE.ProtectBits) == False)
24:         RaiseException(PROTECTION_FAULT);
25:     else
26:         TLB_Insert(VPN, PTE.PFN , PTE.ProtectBits)
27:         RetryInstruction()
```

TLB and Multi-level Page Tables: Summary

■ Translation Lookaside Buffer (TLB)

- A small, fully associative hardware **cache for virtual-to-physical translations**.
- CISC (x86): hardware-managed TLB (hardware page-table walk).
- RISC (MIPS, SPARC): software-managed TLB (TLB miss → trap → OS handler).

■ Multi-level Page Tables

- Split linear page table into page-sized chunks.
- Allocate only the portions that contain valid entries.
- Use a **page directory (PDEs)** to track which page-table pages exist.
- Saves memory by allocating page-table space proportional to usage.
- Trade-off: More page-table levels = more memory references (latency).